

```
!pip install -q requests torch bitsandbytes transformers sentencepiece
accelerate openai httpx==0.27.2
```

```
# imports
```

```
import os
import requests
from IPython.display import Markdown, display, update_display
from openai import OpenAI
from google.colab import drive
from huggingface_hub import login
from google.colab import userdata
from transformers import AutoTokenizer, AutoModelForCausalLM,
TextStreamer, BitsAndBytesConfig
import torch
```

Hàm/Dòng lệnh	Chức năng
!pip install -q requests torch bitsandbytes transformers sentencepiece accelerate openai httpx==0.27.2	Cài đặt các thư viện cần thiết:
requests	Hỗ trợ gửi HTTP request.
torch	Thư viện deep learning PyTorch.
bitsandbytes	Thư viện giúp tối ưu hóa bộ nhớ khi chạy mô hình ngôn ngữ lớn.
transformers	Thư viện của Hugging Face để làm việc với các mô hình NLP.
sentencepiece	Hỗ trợ tokenizer cho các mô hình NLP.
accelerate	Tối ưu hóa tốc độ chạy mô hình trên phần cứng (GPU, TPU, v.v.).
openai	Thư viện giao tiếp với API OpenAI.
httpx==0.27.2	Phiên bản cụ thể của HTTPX để tránh lỗi khi sử dụng OpenAI API.

```
AUDIO_MODEL = "whisper-1"
LLAMA = "meta-llama/Meta-Llama-3.1-8B-Instruct"
```

```
drive.mount("/content/drive")
```

```
audio_filename = "/content/drive/MyDrive/llms/denver_extract.mp3"
```

```
import openai
```

```
openai.api_key = "AIzaSyD41Wx1u0W9KNNM500xbr3MCKD-W-0924A"
```

```
login('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',
add_to_git_credential=True)
```

Hàm/Dòng lệnh	Chức năng
AUDIO_MODEL = "whisper-1"	Định nghĩa mô hình chuyển giọng nói thành văn bản (ASR) Whisper của OpenAI.
LLAMA = "meta-llama/Meta-Llama-3.1-8B-Instruct"	Xác định mô hình ngôn ngữ Meta LLaMA 3.1 8B.
drive.mount("/content/drive")	Gắn kết Google Drive vào Google Colab để truy cập tệp.

audio_filename = "/content/drive/MyDrive/llms/denver_extract.mp3"	Định nghĩa đường dẫn tệp âm thanh cần xử lý.
import openai	Nhập thư viện OpenAI để sử dụng API của OpenAI.
openai.api_key = "AIzaSyD41Wx1u0W9KNNM50Oxbr3MCKD-W-0924A"	(Cảnh báo: Đây là khóa API, không nên chia sẻ công khai vì lý do bảo mật.)
login('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', add_to_git_credential=True)	Đăng nhập vào Hugging Face bằng API Token, đồng thời lưu thông tin vào Git credentials để sử dụng lại sau mà không cần đăng nhập lại.

<pre> system_message = "You are an assistant that produces minutes of meetings from transcripts, with summary, key discussion points, takeaways and action items with owners, in markdown." user_prompt = f"Below is an extract transcript of a Denver council meeting. Please write minutes in markdown, including a summary with attendees, location and date; discussion points; takeaways; and action items with owners.\n{transcription}" messages = [{"role": "system", "content": system_message}, {"role": "user", "content": user_prompt}] quant_config = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=torch.bfloat16, bnb_4bit_quant_type="nf4") </pre>	
Hàm/Dòng lệnh	Chức năng
system_message = "You are an assistant that produces minutes of meetings from transcripts, with summary, key discussion points, takeaways and action items with owners, in markdown."	Xác định thông điệp hệ thống để hướng dẫn trợ lý AI tạo biên bản cuộc họp từ bản ghi.
user_prompt = f"Below is an extract transcript of a Denver council meeting. Please write minutes in markdown, including a summary with attendees, location and date; discussion points; takeaways; and action items with owners.\n{transcription}"	Tạo lời nhắc cho người dùng, yêu cầu AI tạo biên bản cuộc họp từ nội dung bản ghi cuộc họp.
messages = [{"role": "system", "content": system_message}, {"role": "user", "content": user_prompt}]	Định nghĩa danh sách tin nhắn làm đầu vào cho mô hình ngôn ngữ, bao gồm hướng dẫn hệ thống và yêu cầu của người dùng.
quant_config = BitsAndBytesConfig(...)	Cấu hình nén mô hình để giảm dung lượng bộ nhớ, giúp tải mô hình vào GPU dễ dàng hơn.

```
tokenizer = AutoTokenizer.from_pretrained(LLAMA)
tokenizer.pad_token = tokenizer.eos_token
inputs = tokenizer.apply_chat_template(messages,
return_tensors="pt").to("cuda")
streamer = TextStreamer(tokenizer)
model = AutoModelForCausalLM.from_pretrained(LLAMA, device_map="auto",
quantization_config=quant_config)
outputs = model.generate(inputs, max_new_tokens=2000, streamer=streamer)
```

Hàm/Dòng lệnh	Chức năng
tokenizer = AutoTokenizer.from_pretrained(LLAMA)	Tải bộ tokenizer của mô hình LLAMA từ Hugging Face Hub.
tokenizer.pad_token = tokenizer.eos_token	Gán token kết thúc câu (eos_token) làm token đệm (pad_token).
inputs = tokenizer.apply_chat_template(messages, return_tensors="pt").to("cuda")	Áp dụng mẫu hội thoại lên messages, chuyển đổi sang tensor PyTorch và đưa vào GPU.
streamer = TextStreamer(tokenizer)	Tạo bộ phát trực tiếp (streamer) để hiển thị kết quả sinh đầu ra theo luồng.
model = AutoModelForCausalLM.from_pretrained(LLAMA, device_map="auto", quantization_config=quant_config)	Tải mô hình LLAMA, tự động ánh xạ lên GPU và áp dụng cấu hình lượng tử hóa để tiết kiệm bộ nhớ.
outputs = model.generate(inputs, max_new_tokens=2000, streamer=streamer)	Sinh văn bản đầu ra với tối đa 2000 tokens, sử dụng streamer để phát trực tiếp kết quả.

```
response = tokenizer.decode(outputs[0])
display(Markdown(response))
```

Hàm/Dòng lệnh	Chức năng
response = tokenizer.decode(outputs[0])	Giải mã chuỗi token đầu ra (outputs[0]) thành văn bản gốc.
display(Markdown(response))	Hiển thị văn bản đã giải mã dưới dạng Markdown để dễ đọc trong môi trường Notebook.

```

AUDIO_MODEL = "openai/whisper-medium"
speech_model = AutoModelForSpeechSeq2Seq.from_pretrained(AUDIO_MODEL,
torch_dtype=torch.float16, low_cpu_mem_usage=True, use_safetensors=True)
speech_model.to('cuda')
processor = AutoProcessor.from_pretrained(AUDIO_MODEL)

pipe = pipeline(
    "automatic-speech-recognition",
    model=speech_model,
    tokenizer=processor.tokenizer,
    feature_extractor=processor.feature_extractor,
    torch_dtype=torch.float16,
    device='cuda',
)

result = pipe(audio_filename)

transcription = result["text"]
print(transcription)

```

Hàm/Dòng lệnh	Chức năng
AUDIO_MODEL = "openai/whisper-medium"	Khai báo mô hình nhận diện giọng nói tự động (ASR) Whisper Medium của OpenAI.
speech_model = AutoModelForSpeechSeq2Seq.from_pretrained(AUDIO_MODEL, ...)	Tải mô hình nhận diện giọng nói từ thư viện Hugging Face với các cấu hình tối ưu hóa.
speech_model.to('cuda')	Đưa mô hình lên GPU để tăng tốc xử lý.
processor = AutoProcessor.from_pretrained(AUDIO_MODEL)	Tải bộ tiền xử lý dữ liệu âm thanh từ mô hình.
pipe = pipeline("automatic-speech-recognition", ...)	Tạo pipeline xử lý nhận diện giọng nói tự động với mô hình và bộ tiền xử lý.
result = pipe(audio_filename)	Thực thi nhận diện giọng nói trên tệp âm thanh đã chỉ định.
transcription = result["text"]	Lấy văn bản được chuyển đổi từ giọng nói.
print(transcription)	Hiển thị văn bản đã chuyển đổi ra màn hình.